

Lab 1 - Union Find

Course: CIS 387 - Analysis of Algorithms - Fall 2026

Instructor: Dr. Aaron Mininger

1. Getting Set Up

The first step is to clone the course repository into a directory of your choice.

```
git clone https://github.com/amininger-mu/CIS387_F26_Code
```

Then, open the **CIS387_F26_Code** directory in VSCode (Make sure to trust the project). In the Terminal, **cd labs/lab1_unionfind**. (Run all commands from within that directory).

2. Quick-Find Algorithm

Review the code in **QuickFindUF.java** as discussed in class. Then compile:

```
javac IUnionFind.java QuickFindUF.java
```

Now, open the file **DoublingTest.java** and read over the code. It starts by initializing a **QuickFindUF** algorithm with size $N = 100$, and then adds N random edges via union. The program measures the time it took (in microseconds), then doubles the size N and repeats.

Run **DoublingTest.java** and record the following information (it will take around 30 seconds):

2A. What was the maximum size N that it reached, and how long did it take?

2B. Notice the Ratio column, which says how many times longer each iteration took than the previous (with problem size N doubling each time). What do you notice?

3. Quick-Union Algorithm

Review **QuickUnionUF.java** and compile it: **javac QuickUnionUF.java**

Now, open the file **QuickUnionAnalysis.java** and read over the code. It does 3 types of tests: adding all forward edges ($i \rightarrow i+1$), adding all backward edges ($i \rightarrow i-1$), and adding $N-1$ random edges. For each test, it prints out the average tree depth across all sites.

3A. Run **QuickUnionAnalysis** and summarize what you observe in your results. What does this imply about the runtime of **find**?

Now edit **DoublingTest.java** and change line 9 so that it uses the **QuickUnionUF** algorithm:

```
IUnionFind uf = new QuickUnionUF(N);
```

Run **DoublingTest.java** and record the following information:

3B. What was the maximum size N that it reached, and how long did it take?

3C. How do your results compare to those in part 2B?

4. Weighted Quick-Union Algorithm

Review **WeightedQuickUnionUF.java** and compile it: **javac WeightedQuickUnionUF.java**

Modify the code in **QuickUnionAnalysis.java** so that it uses the **WeightedQuickUnionUF** algorithm.

Run the program and answer the following:

4A. Summarize what you observe in your results. How do they compare to the regular Quick-Union results from 3A?

Modify **DoublingTest** to use **WeightedQuickUnionUF** (line 9), then run it.

4B. What was the maximum size N that it reached, and how long did it take?

4C. How do your results compare to those in 2B and 3C? Pay particular attention to the Ratio columns.